Kawi Character Recognition on Copper Inscription Using YOLO Object Detection

Rachmat Santoso
Department of Electrical Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
rachmat.18071@mhs.its.ac.id

Yoyon Kusnendar Suprapto
Department of Electrical Engineering
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
yoyonsuprapto@ee.its.ac.id

Eko Mulyanto Yuniarno
Department of Electrical Engineering
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
ekomulyanto@ee.its.ac.id

***Abstract*— Inscriptions are an important source for learning historical events that occurred in the past. Since the middle of the 8th century AD, in the era of the Indonesian kingdom, inscriptions were written using Kawi character. The problem of reading Kawi character on inscriptions occurs when the media used is experiencing interference. This research focuses on copper inscriptions and uses YOLO object detection to recognize Kawi character on copper inscriptions, especially those with *patina*. Dataset used for the training in this research were 1518 images (1311 train, 207 validation) that comes from 69 original images of Waruṅgahan Inscription using data augmentation. This research using 657 classes in total. In the training, tested on the validation images, the model obtained a mAP of 97.17% (threshold = 0.4). In the test, tested on test images, the model recognized Kawi character on the Waruṅgahan Inscriptions properly, with an accuracy of 97.93% (threshold = 0.4).**

Keywords — kawi character, waruṅgahan inscription, yolo object detection

I. INTRODUCTION

Inscriptions are an important historical source for learning events that occurred in the past since they contain information about the time, chronology, names of figures, and the reasons why the inscriptions were made. Inscriptions are written in various media such as “lontar”, copper, and stone. Since the middle of the 8th century AD, in the era of the Indonesian kingdom, inscriptions were written using Kawi character or also known as ancient Javanese character. Kawi character is widely used in Java and Bali to write in Sanskrit and ancient Javanese. Different from Carakan character (new Javanese character or "hanacaraka"), Kawi character has 33 consonant characters, 16 vowel characters, number characters, and punctuation characters. Compared to Carakan character, writing and reading Kawi character is more difficult to do. This is because Kawi character has complex shape, has ligature characters, as well as has particular rules for reading the characters. An article [1] reminded the importance of inscriptions to be learned since they contain many historical knowledge which is still relevant to the current situation. Unfortunately, today, reading Kawi character on inscriptions is more difficult to do, considering the age of inscriptions as well as the depth and location where the inscriptions was found. The problem of reading Kawi character on inscriptions occurs when the media used is experiencing interference, such as red spots on “lontar”, *patina* on copper, and erosion or broken carvings on stone. Therefore, Kawi character recognition on inscriptions must be a particular concern. Besides aiming to help historical researchers, it also helps the public to recognize Kawi character on inscriptions.

Based on the issues mentioned, this research will perform Kawi character recognition on inscriptions, especially inscriptions whose characters become difficult to read due to interference. The recognition of Kawi character on inscriptions will be performed using YOLO (You Only Look Once) object detection, which implements the architecture of CNN (Convolutional Neural Network). CNN [2] [3] is one of the artificial neural network architectures that commonly used in image data to perform image recognition such as classification, OCR (Optical Character Recognition), object detection, etc. YOLO is used in this research because of its high detection accuracy and high detection speed when detecting objects in an image. YOLO outperforms other object detection methods that are also widely used, as in [4] [5] [6]. In addition, another important ability is that YOLO is able to detect multiple objects in an image simultaneously. This ability will be effective when used to recognize Kawi character on inscriptions, considering on the inscriptions there are many characters that must be recognized.

This research focus on recognizing Kawi character on inscriptions with copper media. Since most of the copper inscriptions found have a *patina*, this cause reading the characters relatively difficult to do. Dataset used in this research was taken from the original Waruṅgahan Inscription. This inscription is one of the copper media inscriptions from 1227 Śaka or 1305 AD that made during the era of Majapahit Kingdom which was led by King Śrī Maharaja Nararyya Sanggramawijaya, as in [7]. Majapahit Kingdom is one of the kingdoms that existed during the kingdom era in Indonesian history.

Several research related to character recognition using CNN architecture have been performed. Rismiyati, et al. [8] performed research on recognition of handwriting Carakan character, which consists of 20 characters with paper media. The research compared two methods for recognizing handwriting Carakan character, namely CNN and DNN (Deep Neural Network) with accuracy results of 70.22% and 64.41%, respectively. The results of this research indicates that CNN has better accuracy. Then, research from Chomphuwiset [9], applied CNN to segmentation and recognition of Thai characters. The dataset used consists of Thai characters and some special characters, which are divided into 66 classes. The result is an accuracy of 98%. Other research by Alizadehashrafi, et al. [10] performed handwritten Persian character recognition using CNN. The dataset used for this research is Hoda's handwritten which is the most widely used dataset and the largest dataset. This research implemented two types of CNN, namely SCNN and ECNN, the results were 96.3% and 97.1%, respectively. The

978-1-7281-8283-4/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: Rutgers University. Downloaded on May 19,2021 at 09:39:00 UTC from IEEE Xplore. Restrictions apply.

research above prove that CNN can provide high accuracy for character recognition.

Using YOLO in this research, it is expected to provide high accuracy as other research that also apply CNN architecture. Such as other object detection methods, YOLO is expected to be able to detect all Kawi characters on the Waruṅgahan Inscription as well as to show its position in the image by providing a bounding box for each character as well as a label for the reading purpose. This research will use a version of YOLO object detection, namely YOLOv3-tiny.

The rest of the paper is as follows: Section 2 describes the YOLO object detection. Section 3 describes the methodology. Section 4 discusses the research results. Finally, conclusions and future work are presented in section 5.

II. YOLO Object Detection

YOLO is an object detection method that has high object detection speed and high accuracy. It sees the entire image during training and test, this makes it possible to encode contextual information about the classes properly. YOLO also learns the generalizable representations of objects. Using only one evaluation, YOLO with its system can simultaneously detect all objects contained in an image.

In YOLOv1, each grid cell can only detect one object whose center falls into the grid cell. This limits the number of nearby objects that can be detected, especially small objects that appear in groups. Overcoming this limitation, YOLOv2 and YOLOv3 use anchor boxes that allow the system to detect multiple objects per grid cell. Even though the center of two different objects fall on the same grid cell, those two objects can still be detected.

As mentioned before, this research uses YOLOv3-tiny. But before discussing about YOLOv3-tiny, it is important to know how YOLOv3 works as well as to find out the differences compared to previous versions of YOLO.

A. YOLOv3

YOLOv3 [6] is an improvement over the previous version, YOLO (YOLOv1) [4] and YOLOv2 [5]. YOLOv3 uses darknet-53, a feature extractor that has 53 convolution layers. It is a combined approach between Darknet-19 and residual network. It uses 3×3 and 1×1 filters and several shortcut connections as in residual neural network (ResNet) [11]. Using Darknet-53 feature extractor, YOLOv3 obtain deeper feature information. Darknet-53 is shown in Fig. 1. It is important to note, since Darknet-53 will be used as a feature extractor then Avgpool, Connected, and Softmax in Fig. 1 are not included. Those three layers will only be included when Darknet-53 is used as a classifier on Imagenet.

On class prediction, YOLOv3 does not use a softmax, instead it uses independent logistic classifiers for all classes. This allows YOLOv3 to perform multilabel classification. This means that one object can have more than one class prediction at the same time e.g. an object can be detected as a male and as a person at the same time. During training YOLOv3 uses the binary cross entropy loss for class predictions.

	TYPE	FILTERS	SIZE	OUTPUT
	CONVOLUTIONAL	32	3 x 3	256 x 256
	CONVOLUTIONAL	64	3 x 3 / 2	128 x 128
1x	CONVOLUTIONAL	32	1 x 1	
	CONVOLUTIONAL	64	3 x 3	
	RESIDUAL			128 x 128
	CONVOLUTIONAL	128	3 x 3 / 2	64 x 64
2x	CONVOLUTIONAL	64	1 x 1	
	CONVOLUTIONAL	128	3 x 3	
	RESIDUAL			64 x 64
	CONVOLUTIONAL	256	3 x 3 / 2	32 x 32
8x	CONVOLUTIONAL	128	1 x 1	
	CONVOLUTIONAL	256	3 x 3	
	RESIDUAL			32 x 32
	CONVOLUTIONAL	512	3 x 3 / 2	16 x 16
8x	CONVOLUTIONAL	256	1 x 1	
	CONVOLUTIONAL	512	3 x 3	
	RESIDUAL			16 x 16
	CONVOLUTIONAL	1024	3 x 3 / 2	8 x 8
4x	CONVOLUTIONAL	512	1 x 1	
	CONVOLUTIONAL	1024	3 x 3	
	RESIDUAL			8 x 8
	AVGPOOL		GLOBAL	
	CONNECTED		1000	
	SOFTMAX			

Fig. 1. Darknet-53 (source: YOLOv3 paper)

YOLOv3 performs predictions at 3 different scales to accommodate different objects size. Extracting features at the scales uses a concept similar to FPN (Feature Pyramid Network) [12]. YOLOv3 uses different strides with sizes 32, 16, 8. This means, if the input image is 416×416, YOLOv3 will perform detection on the scales 13×13, 26×26, and 52×52.

When performing detection, YOLOv3 divides the image into $S \times S$ grid e.g 13×13 grid on the first scale. Each grid cell predicts bounding boxes and objectness, as well as predicts classes. YOLOv3 use k-means clusterring to determine anchor boxes as in previous versions. On each scale, YOLOv3 uses 3 anchor boxes to predict 3 bounding boxes per grid cell. Each bounding box has form (b_x, b_y, b_w, b_h) as in (1).

$$\begin{aligned} b_x &= \sigma(t_x) + c_x \\ b_y &= \sigma(t_y) + c_y \\ b_w &= p_w e^{t_w} \\ b_h &= p_h e^{t_h} \end{aligned} \tag{1}$$

, where (b_x, b_y) denotes the center of the bounding box; (b_w, b_h) denotes the width and height of the bounding box; (p_w, p_h) denotes the width and height of the anchor box; (t_x, t_y, t_w, t_h) is the predicted result of the network; and (c_x, c_y) denotes the top left corner of the image.

To predict the objectness score for each bounding box, YOLOv3 uses logistic regression. The objectness score represents the probability that an object is inside the bounding box. If the bounding box prior overlaps the ground truth of the object more than the others, then the objectness score should be nearly 1.

During training, YOLOv3 optimized the multi-part loss function: coordinate prediction error, confidence score / IOU error, and classification error. The box size error uses MSE,

Authorized licensed use limited to: Rutgers University. Downloaded on May 19,2021 at 09:39:00 UTC from IEEE Xplore. Restrictions apply.

while the other errors use the binary cross entropy loss (bce) (2).

$$bce(z,\hat{z}) = -(z \cdot \log(\hat{z}) + (1-z) \cdot \log(1-\hat{z})) \quad (2)$$

, where z denotes the target value; $\hat{z}$ denotes the predicted value.

YOLOv3 loss function (3), as follows:

$$\begin{aligned} Loss = {} & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[bce(x_i,\hat{x}_i) + bce(y_i,\hat{y}_i) \right] \\ & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[\left(\sqrt{w}_i - \sqrt{\hat{w}_i}\right)^2 + \left(\sqrt{h}_i - \sqrt{\hat{h}_i}\right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} bce(C_i,\hat{C}_i) + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{noobj} bce(C_i,\hat{C}_i) \\ & + \sum_{i=0}^{S^2} \mathbb{1}_{i}^{obj} \sum_{c \,\epsilon\, classes} bce(p_i(c),\hat{p}_i(c)) \end{aligned} \quad (3)$$

, where S^2 denotes the number of grids; B denotes the number of bounding boxes per grid; i, j are iterators for cell and bounding box, respectively; $\mathbb{1}_i^{obj}$ denotes object appears in cell i ; $\mathbb{1}_{ij}^{obj}$ denotes that j^{th} bounding box handle prediction of object that appears in cell i ; $\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i$ denotes the predicted center coordinates and the size of the bounding box; x_i, y_i, w_i, h_i are the target / ground truth values; $C\hat{\imath}$ denotes predicted confidence; C_i denotes real data; $\hat{p}_i(c), p_i(c)$ represent predicted class probabilities and true values, respectively; λ_{coord} and λ_{noobj} are two coefficients used to prevent model instability that causes training to diverge early on.

On the COCO dataset, using an image size of 416×416, YOLOv3 predicts $((13 \times 13) + (26 \times 26) + (52 \times 52)) \times 3 = 10647$ bounding boxes. Hence, an object may be detected by multiple bounding boxes. To ensure that every object in the image is detected only by the best bounding box, YOLOv3 uses NMS (Non-Max Suppession). The NMS will remove all bounding boxes that have confidence below the specified threshold (e.g threshold = 0.4). Then choose the remaining bounding box that has the highest confidence and then discard the remaining redundant bounding boxes again. The bounding box that has the highest confidence is used to detect objects.

B. YOLOv3-tiny

YOLOv3-tiny is a light version of YOLOv3. It still uses the same principles as YOLOv3 but with some changes. YOLOv3-tiny reduces the number of layers used. YOLOv3 enlarge stride of the convolutional kernel to reduce the size of the features map, whereas YOLOv3-tiny uses the pooling layer to do so. When performing detection, YOLOv3-tiny only uses 2 scales that obtained by applying stride with sizes 32, 16. The loss function and the number of bounding boxes per grid used are still the same as YOLOv3. YOLOv3-tiny architecture is shown in Fig. 5.

As previously mentioned, YOLOv3-tiny will be used in this research. As in [3], the CNN layer works with a hierarchy, the initial/ low layer learns simple features, the next layer learns more complex features of objects than before. This means that the deeper the layers used, the features that are extracted from an object are more complex or have a higher level. Based on that theory, since the data used in this research are character, which is a combination of simple features such as lines, curves, and blobs, thus there is not really necessary to use a large number of layers. Using YOLOv3-tiny on data in the form of character, it is expected that the model will obtain high detection accuracy and high detection speed. Using the anchor box mechanism with 3 bounding boxes per grid, it is expected that all Kawi characters on the inscription can be recognized properly, considering the position of the characters are adjacent.

III. METHODOLOGY

A. Dataset

The dataset used in this research was taken from the original Waruṅgahan inscription. All characters on the inscription will be used, such as consonants, vowels, numbers, symbols, punctuation, as well as their combinations. The Waruṅgahan Inscription consists of 14 plates (11 plates with characters on both sides, 1 plate with characters on one side, and 2 blank plates). The data used is a piece of the original plate of the Waruṅgahan inscription, with a total of 69 images. This research will use grayscale images during the training process, considering that the RGB color in type of dataset of Kawi character can be ignored. Using grayscale images certainly reduce computation cost during the training process since it only uses images with only one channel. Fig. 2 shows pieces of the original Waruṅgahan Inscription plates.



Fig. 2. Pieces of the Original Waruṅgahan Inscription Plate (source: G. A. Sambodo and W. P. Sidhi, members of *Tapak Jejak Kerajaan* community)

- *Data Augmentation*

Experiments conducted by Shijie, et al. [13] and Gandhi [14] show that the application of data augmentation to the dataset increase the accuracy of the network model. Based on the experimental results, this research uses data augmentation

Authorized licensed use limited to: Rutgers University. Downloaded on May 19,2021 at 09:39:00 UTC from IEEE Xplore. Restrictions apply.

on the original dataset to increase the number of dataset as well as increase the variation of the dataset used for training. The purpose is to acquire a network model that has high accuracy

This research uses data augmentation methods that commonly used, such as translation, rotation, scale, gaussian blur, and their combinations. Each image in the original dataset will generate at least 21 new images after augmentation. The data augmentation methods used in this research is shown in Table 1. Examples of data augmentation results on one of the images in the original dataset are shown in Fig. 3.

After augmentation, total of the dataset in this research became 1518 images, each image has approximately 90 characters. For training purposes this dataset is divided into 1311 train images and 207 validation images.

TABLE 1 DATA AUGMENTATION METHODS

Data Augmentation			
No.	**Method**	**No.**	**Method**
1	Translate-right 120 pixels	13	Translate-right 120 pixels, zoom in
2	Translate-up 120 pixels	14	Translate-up 120 pixels, zoom in
3	Translate-left 120 pixels	15	Translate-left 120 pixels, zoom in
4	Translate-down 120 pixels	16.	Translate-down 120 pixels, zoom in
5	Translate-right 120 pixels, rotate 2 degree CW	17	Translate-right 120 pixels, zoom out
6	Translate-up 120 pixels, rotate 2 degree CW	18	Translate-up 120 pixels, zoom out
7	Translate-left 120 pixels, rotate 2 degree CW	19	Translate-left 120 pixels, zoom out
8	Translate-down 120 pixels, rotate 2 degree CW	20	Translate-down 120 pixels, zoom out
9	Translate-right 120 pixels, rotate 2 degree CCW	21	Gaussian blur
10	Translate-up 120 pixels, rotate 2 degree CCW		
11	Translate-left 120 pixels, rotate 2 degree CCW		
12	Translate-down 120 pixels, rotate 2 degree CCW		

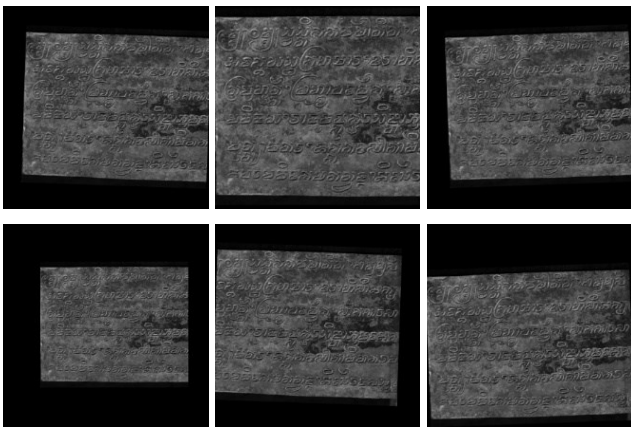


Fig. 3. Examples of Data Augmentation Results

- *Labeling*

Labeling is an important process of this research. It must be ensured that every character in the dataset is labeled correctly. The purpose of labeling is to mark the original area of the object (ground truth) which will be used to train the network model. Labeling was carried out on 1518 images using *labelImg*. The results are stored in a *.txt* file which consists of five values representing class, x, y, width, and height.

Based on the calculation of the number of labels from all *.txt* files, there are 137490 labels in total, with the number of classes used is 657 classes. The classes for this labeling are stored in the *classes.txt* file. Fig. 4 shows some examples of the classes used in this research.

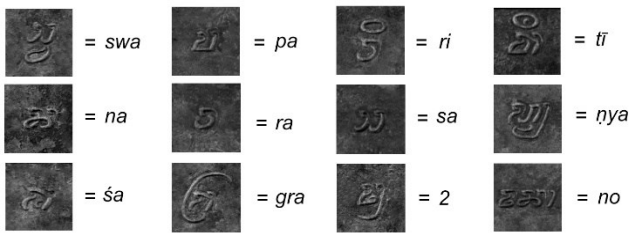


Fig. 4. Examples of Classes

B. Framework

This research uses YOLOv3-tiny which is run on the open source neural network framework, Darknet [15]. Images used for training are grayscale images (1 channel) with a size of 832×832. The *flip* parameter for YOLOv3-tiny default data augmentation is set to 0, considering that the dataset in this research uses Kawi character that cannot be reversed. *angle* is set to 0 since this research uses its own degree value for augmentation. The initial leaning rate is 0.001, the value will gradually decrease as the iteration increases. The maximum iteration used is 130000. The number of filters on the convolution layer before the yolo layer is set to 1986, is obtained from $anchor \times (5 + number\ of\ class) = 3 \times (5 + 657)$. The number of classes on each yolo layer is set to 657. The rest of the settings follow the default yolov3-tiny.cfg. Fig. 5 shows the YOLOv3-tiny architecture.

	LAYER	FILTERS	SIZE/ STRIDE	INPUT		OUTPUT
0	convolutional	16	3 x 3/ 1	832 x 832 x 1	->	832 x 832 x 16
1	max-pool		2x 2/ 2	832 x 832 x 16	->	416 x 416 x 16
2	convolutional	32	3 x 3/ 1	416 x 416 x 16	->	416 x 416 x 32
3	max-pool		2x 2/ 2	416 x 416 x 32	->	208 x 208 x 32
4	convolutional	64	3 x 3/ 1	208 x 208 x 32	->	208 x 208 x 64
5	max-pool		2x 2/ 2	208 x 208 x 64	->	104 x 104 x 64
6	convolutional	128	3 x 3/ 1	104 x 104 x 64	->	104 x 104 x 128
7	max-pool		2x 2/ 2	104 x 104 x 128	->	52 x 52 x 128
8	convolutional	256	3 x 3/ 1	52 x 52 x 128	->	52 x 52 x 256
9	max-pool		2x 2/ 2	52 x 52 x 256	->	26 x 26 x 256
10	convolutional	512	3 x 3/ 1	26 x 26 x 256	->	26 x 26 x 512
11	max-pool		2x 2/ 1	26 x 26 x 512	->	26 x 26 x 512
12	convolutional	1024	3 x 3/ 1	26 x 26 x 512	->	26 x 26 x 1024
13	convolutional	256	1 x 1/ 1	26 x 26 x 1024	->	26 x 26 x 256
14	convolutional	512	3 x 3/ 1	26 x 26 x 256	->	26 x 26 x 512
15	convolutional	1986	1 x 1/ 1	26 x 26 x 512	->	26 x 26 x 1986
16	yolo					
17	route	13			->	26 x 26 x 256
18	convolutional	128	1 x 1/ 1	26 x 26 x 256	->	26 x 26 x 128
19	upsample		2x	26 x 26 x 128	->	52 x 52 x 128
20	route	19 8			->	52 x 52 x 384
21	convolutional	256	3 x 3/ 1	52 x 52 x 384	->	52 x 52 x 256
22	convolutional	1986	1 x 1/ 1	52 x 52 x 256	->	52 x 52 x 1986
23	yolo					

Fig. 5. YOLOv3-tiny Architecture (source : Darknet)

IV. Result

A. Training Result

At iteration 130000, using threshhold = 0.4, the model obtained a mAP (mean Average Precision) of 97.17%. mAP shows how well the model detects objects, the higher the mAP means the more accurate the model. Based on the mAP value, it means that the YOLOv3-tiny model has a high performance in detecting Kawi character on the Waruṅgahan Inscription. Fig. 6 shows the mAP during training.

Authorized licensed use limited to: Rutgers University. Downloaded on May 19,2021 at 09:39:00 UTC from IEEE Xplore. Restrictions apply.

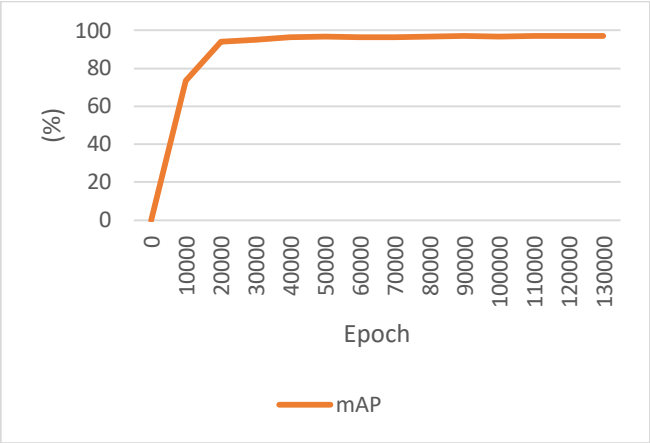


Fig. 6. mAP during Training

B. Test

Aims to know the robustness of the model detecting Kawi character on the Waruṅgahan Inscription, then the model was tested on the test images. This test uses 23 test images, taken randomly from each side of the Waruṅgahan Inscription plate that contains characters. Then, the test images were augmented using different methods from the training dataset i.e. using different translation direction and different degree values, adjust hue/ saturation, and adjust exposure. The test results are shown in Fig. 7. The accuracy (ACC) of the test is calculated using (4).

$$ACC = \left(\frac{TP + TN}{TP + TN + FP + FN}\right) x\ 100\% \qquad (4)$$

, where TP (True Positive), TN (True Negative), FP (False Positive), dan FN (False Negative).

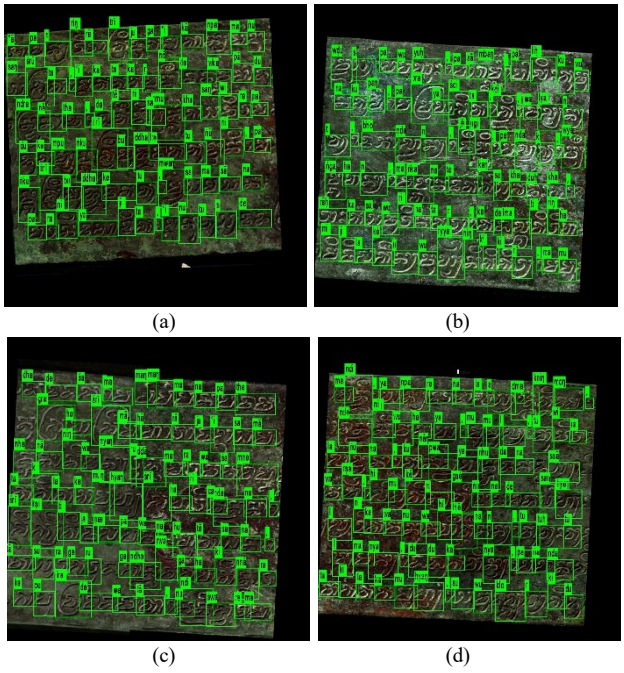


Fig. 7. Examples of Test Results (threshold = 0.4, the model will ignore detection when its confidence value is below 0.4): (a) test_img_4 (accuracy = 100 %); (b) test_img_14 (accuracy = 98.83 %); (c) test_img_21 (accuracy = 96.51 %); (d) test_img_23 (accuracy = 97.82 %)

Compared with ground truth and calculated using (4), the accuracy of each test result is shown in Table 2.

TABLE 2 ACCURACY OF TEST RESULTS

No.	Name	ACC (%)	No.	Name	ACC (%)
1	test_img_1	96.25	13	test_img_13	95.34
2	test_img_2	98.71	14	test_img_14	98.83
3	test_img_3	97.4	15	test_img_15	98.83
4	test_img_4	100	16	test_img_16	94.73
5	test_img_5	98.71	17	test_img_17	98.96
6	test_img_6	98.68	18	test_img_18	97.7
7	test_img_7	98.59	19	test_img_19	98.86
8	test_img_8	98.68	20	test_img_20	98.80
9	test_img_9	100	21	test_img_21	96.51
10	test_img_10	96	22	test_img_22	98.91
11	test_img_11	98.7	23	test_img_23	97.82
12	test_img_12	97.4			
Average Accuracy (%)			**97.93**		

The results show that YOLOv3-tiny model provides high accuracy when detecting Kawi character on the images. However, there are still some undetected characters. This is due to several factors, first, the characters near the undetected characters are missing due to image cropping. Referring YOLO paper [4], YOLO object detection uses features from the entire image when making predictions. This means YOLO not only sees features of the objects but also contextual information about the objects, such as the position of the objects in the image and other objects around them. In the test results, the missing characters due to cropping causes confidence score of the character detection is decreased. Unfortunately, the confidence score decreased below the specified threshold, hence the detection results were ignored. This issue can be accomplished by adding characters around the undetected characters to increase the confidence score or by taking another piece of image to detect the undetected characters.

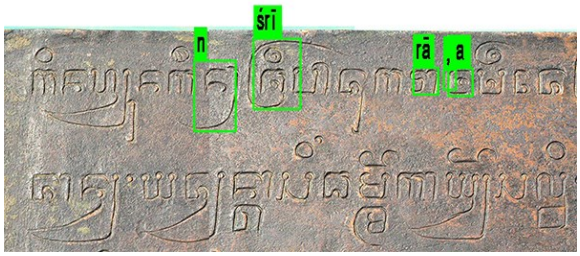
Second, due to the lack of variation in data augmentation. In Fig. 7, if the augmentation used in the test image is changed e.g. change the direction of rotation, then the undetected characters will be detected. But on the other hand, due to changing augmentation, the confidence score of certain characters may decrease below the specified threshold which can make detection ignored. Therefore, it is necessary to add more augmentation with different methods to increase the robustness of the model.

C. Additional Test

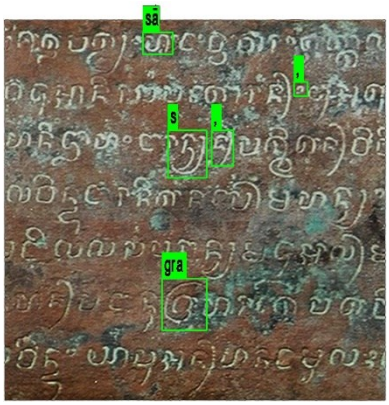
Fig. 8 shows the test results on pieces of other inscriptions, namely the Adan-Adan Inscription and the Munduan Inscription. The results show that many characters are not detected and some characters are detected incorrectly. This shows that the model that is trained using the characters from the Waruṅgahan Inscription was not effective for detecting characters from other inscriptions, even though the Adan-Adan Inscription and the Waruṅgahan Inscription were made in the same era. This ineffectiveness is due to the different shape/ writing style of the Kawi character they have. Since the data used are characters, the difference in the

Authorized licensed use limited to: Rutgers University. Downloaded on May 19,2021 at 09:39:00 UTC from IEEE Xplore. Restrictions apply.

combination between lines and curves can be sensitive. It is important to know that although the inscriptions are written using Kawi character and have the same basic pattern, however the shape/ writing style of Kawi character used in every era has differences. Then, in an era, there were many inscription writers who also had different writing styles. These factors affect the shape/ writing style of Kawi character on the inscription.



(a)



(b)

Fig. 8. Test Results on Pieces of Other Inscriptions Plate: (a) Adan-Adan Inscription. This is another inscription from 1223 Śaka or 1301 AD made during the Majapahit Kingdom era that led by King Śrī Maharaja Nararyya Sanggramawijaya; (b) Munduan Inscription from 728 Śaka or 806 AD

V. Conclusion and Future Work

This research uses YOLOv3-tiny to recognize Kawi character on copper inscriptions, especially the Waruṅgahan Inscription. Before the training process, data augmentation was carried out to increase the number of datasets as well as to increase dataset variations. Tested on test images that take different pieces of inscription that is different from the training dataset and use different augmentation, the model can detect Kawi character on the inscription properly, with an average accuracy of 97.93%. Even though it obtains high accuracy, further research needs to add variations of the data using more data augmentation methods to increase the robustness of the model.

The results of the additional test show that the model trained with the dataset from the Waruṅgahan Inscription is ineffective when used on other inscriptions. This is due to the different shape/ writing style of the Kawi character used on the inscriptions. Therefore, in the future work, it is necessary to add dataset from other inscriptions, so that the model works properly when used to recognize Kawi character from other inscriptions.

Acknowledgment

The authors would like to thank *Tapak Jejak Kerajaan* community dan Mpu Tantular Museum that located at Sidoarjo, East Java, Indonesia that provided a convenient place to learn about Kawi character which was useful for completing this research. All of the original Waruṅgahan Inscription images used in this research belong to G. A. Sambodo and W. P. Sidhi, members of *Tapak Jejak Kerajaan* community.

References

[1] A. P. Anggoro on Kompas.com, “Pelajaran mengendalikan banjir dari klagen,” (online), available: *https://tekno.kompas.com/read/2008/11/15/01212614/pelajaran.mengendalikan.banjir.dari.klagen*, accessed on 10 February 2020.

[2] K. O'Shea and R. Nash, “An introduction to convolutional neural networks,” arXiv preprint arXiv:1511.08458v2, 2015.

[3] Z. Qin, F. Yu, C. Liu, and X. Chen, “How convolutional neural network see the world - A survey of convolutional neural network visualization methods,” arXiv preprint arXiv:1804.11191v2, 2018.

[4] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, 2016, pp. 779-788.

[5] J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, 2017, pp. 6517-6525.

[6] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” arXiv preprint arXiv:1804.02767, 2018.

[7] G. A. Sambodo, “Prasasti Waruṅgahan, sebuah data baru dari masa awal Majapahit,” AMERTA, Journal of Archeology Research and Development, Vol. 36 No. 1, 2018.

[8] Rismiyati, Khadijah and A. Nurhadiyatna, “Deep learning for handwritten Javanese character recognition,” 2017 1st International Conference on Informatics and Computational Sciences (ICICoS), Semarang, 2017, pp. 59-64.

[9] P. Chomphuwiset, “Printed thai character segmentation and recognition,” 2017 IEEE 4th International Conference on Soft Computing & Machine Intelligence (ISCMI), Port Louis, 2017, pp. 123-127.

[10] B. Alizadehashraf and S. Roohi, “Persian handwritten character recognition using convolutional neural network,” 2017 10th Iranian Conference on Machine Vision and Image Processing (MVIP), Isfahan, 2017, pp. 247-251.

[11] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” arXiv preprint arXiv:1512.03385, 2015.

[12] T. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, "Feature pyramid networks for object detection," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, 2017, pp. 936-944.

[13] J. Shijie, W. Ping, J. Peiyi and H. Siping, “Research on data augmentation for image classification based on convolution neural networks,” 2017 Chinese Automation Congress (CAC), Jinan, 2017, pp. 4165-4170.

[14] A. Gandhi on Nanonets.com, “Data augmentation | how to use deep learning when you have limited data—part 2,” (online), Available: https://nanonets.com/blog/data-augmentation-how-to-use-deep-learning-when-you-have-limited-data-part-2/, accessed on 22 March 2020.

[15] J. Redmon, “Darknet: Open source neural networks in c,” http://pjreddie.com/darknet/, 2013–2016.

Authorized licensed use limited to: Rutgers University. Downloaded on May 19,2021 at 09:39:00 UTC from IEEE Xplore. Restrictions apply.